Advanced Library Management System

Project type: DBMS Mini Project

Course: Bachelor of Computer Applications (BCA)

Database: MySQL 8.0+

Database name: `librarydb_advanced`

Software: MySQL Workbench

1. Project overview

The Advanced Library Management System is a relational database project for maintaining library members, books, borrowing transactions, reservations and library activity records. It extends a basic library database with automation and reporting features using SQL.

2. Main modules

- **Member management:** stores student and faculty details and member status.
- **Book management:** stores book details, categories, prices, shelf locations and copy counts.
- **Issue and return management:** records borrowing dates, due dates and returns.
- **Reservation management:** records members waiting for books.
- **Fine reporting:** calculates a fine of ₹2 per late day.
- **Activity tracking:** records issue and return events.

3. Database tables

Table	Purpose
`members`	Student and faculty member details
`categories`	Book category master data
`books`	Book information and copy availability
`issues`	Issue, due and return transactions
`reservations`	Book reservation requests
`activity_log`	Important activity history

4. Relationships

- One category can contain many books.
- One member can have many issue records.
- One book can appear in many issue records over time.
- One member can create many reservations.
- One book can have many reservation records.

The `issues` and `reservations` tables use foreign keys to connect members and books.

5. Business rules

- Student borrowing period: 14 days when the due date is not supplied.
- Faculty borrowing period: 30 days when the due date is not supplied.
- Fine: ₹2 for each day after the due date.
- A book returned on or before its due date has no late fine.
- A book with a `NULL` return date is currently on loan.

- A trigger decreases available copies when a new issue is inserted.
- A trigger increases available copies when an unreturned book is returned.
- A book cannot be issued if it has no available copies or is archived.

6. SQL concepts demonstrated

1. Database and table creation
2. Primary keys and foreign keys
3. `NOT NULL`, `UNIQUE`, `CHECK` and `DEFAULT` constraints
4. `SELECT`, `WHERE`, `ORDER BY` and `LIMIT`
5. Aggregate functions: `COUNT`, `SUM`
6. `GROUP BY` and `HAVING`
7. `INNER JOIN` and `LEFT JOIN`
8. Subqueries
9. `NULL` handling and `COALESCE`
10. Date functions: `DATEDIFF`, `DATE_ADD`
11. Window function: `RANK()`
12. Stored procedures
13. Triggers
14. Views
15. Indexes
16. Transaction activity logging

7. How to run

1. Open MySQL Workbench and connect to your MySQL server.
2. Open `library_management_advanced.sql`.
3. Execute the complete script.
4. The script creates and uses a separate database named `librarydb_advanced`, so it does not overwrite the original `librarydb`.
5. Expand `librarydb_advanced` in the SCHEMAS panel and inspect its tables and views.
6. Run the report queries included at the bottom of the SQL file.
7. To test automation, call a procedure with valid IDs, for example:
   - `CALL sp_issue_book(1, 1, NULL);`
   - Find the new issue ID, then call `CALL sp_return_book(issue_id);`
   - `CALL sp_member_history(1);`

Important: Run the issue and return procedure examples only when you want to create test transactions. They change the sample database.

8. Useful views

- `vw_current_issues`: books that have not yet been returned.
- `vw_fine_report`: late days and calculated fines for returned and currently borrowed books.
- `vw_book_availability`: total copies, available copies and copies on loan.

9. Limitations and future enhancements

This is an educational DBMS project, not a production-ready library application. Possible future improvements include a web interface, librarian login and role permissions, reservation queue processing, fine payment records, email reminders, and stronger transaction handling for concurrent users.

10. Conclusion

The project demonstrates how a relational database can support everyday library operations. By combining normalized tables, constraints, joins, views, stored procedures, triggers and indexes, it provides a more complete example of database design and SQL automation.

Appendix: Complete SQL Script

library_management_advanced.sql

```
-- ============================================================
-- ADVANCED LIBRARY MANAGEMENT SYSTEM
-- DBMS Mini Project | MySQL 8.0+
-- Database: librarydb_advanced
-- Run this script in MySQL Workbench.
-- ============================================================

CREATE DATABASE IF NOT EXISTS librarydb_advanced;
USE librarydb_advanced;

-- 1. MASTER TABLES
CREATE TABLE IF NOT EXISTS members (
    member_id INT PRIMARY KEY AUTO_INCREMENT,
    member_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    phone VARCHAR(20),
    member_type ENUM('Student', 'Faculty') NOT NULL,
    joined_date DATE NOT NULL DEFAULT (CURRENT_DATE),
    member_status ENUM('Active', 'Inactive') NOT NULL DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS categories (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(80) NOT NULL UNIQUE,
    description VARCHAR(255)
);

CREATE TABLE IF NOT EXISTS books (
    book_id INT PRIMARY KEY AUTO_INCREMENT,
    isbn VARCHAR(20) UNIQUE,
    title VARCHAR(180) NOT NULL,
    author VARCHAR(120) NOT NULL,
    category_id INT NOT NULL,
    publisher VARCHAR(120),
    publication_year YEAR,
    price DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    total_copies INT NOT NULL DEFAULT 1,
    copies_available INT NOT NULL DEFAULT 1,
    shelf_location VARCHAR(30),
    book_status ENUM('Active', 'Archived') NOT NULL DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_books_category FOREIGN KEY (category_id)
        REFERENCES categories(category_id),
    CONSTRAINT chk_book_price CHECK (price >= 0),
    CONSTRAINT chk_total_copies CHECK (total_copies >= 0),
    CONSTRAINT chk_available_copies CHECK (
        copies_available >= 0 AND copies_available <= total_copies
    )
);

-- 2. TRANSACTION TABLES
CREATE TABLE IF NOT EXISTS issues (
    issue_id INT PRIMARY KEY AUTO_INCREMENT,
    member_id INT NOT NULL,
    book_id INT NOT NULL,
    issue_date DATE NOT NULL DEFAULT (CURRENT_DATE),
    due_date DATE NOT NULL,
    return_date DATE NULL,
    issue_status ENUM('Issued', 'Returned') NOT NULL DEFAULT 'Issued',
```

```
    CONSTRAINT fk_issues_member FOREIGN KEY (member_id)
        REFERENCES members(member_id),
    CONSTRAINT fk_issues_book FOREIGN KEY (book_id)
        REFERENCES books(book_id),
    CONSTRAINT chk_issue_dates CHECK (due_date >= issue_date),
    INDEX idx_issue_member (member_id),
    INDEX idx_issue_book (book_id),
    INDEX idx_issue_due (due_date)
);

CREATE TABLE IF NOT EXISTS reservations (
    reservation_id INT PRIMARY KEY AUTO_INCREMENT,
    member_id INT NOT NULL,
    book_id INT NOT NULL,
    reservation_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    reservation_status ENUM('Pending', 'Fulfilled', 'Cancelled')
        NOT NULL DEFAULT 'Pending',
    CONSTRAINT fk_reservations_member FOREIGN KEY (member_id)
        REFERENCES members(member_id),
    CONSTRAINT fk_reservations_book FOREIGN KEY (book_id)
        REFERENCES books(book_id),
    INDEX idx_reservation_status (reservation_status)
);

CREATE TABLE IF NOT EXISTS activity_log (
    log_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    activity_type VARCHAR(40) NOT NULL,
    details VARCHAR(500) NOT NULL,
    activity_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. SAMPLE MASTER DATA
INSERT IGNORE INTO categories (category_id, category_name, description) VALUES
(1, 'Programming', 'Programming languages and software development'),
(2, 'Database', 'Database systems and data management'),
(3, 'Networking', 'Computer networks and communication'),
(4, 'Fiction', 'Fiction and literature'),
(5, 'Science', 'General science and technology'),
(6, 'Mathematics', 'Mathematics and statistics');

INSERT IGNORE INTO members
(member_id, member_name, email, phone, member_type, joined_date) VALUES
(1, 'Aarav Kumar', 'aarav@example.com', '9000000001', 'Student', '2026-01-10'),
(2, 'Diya Sharma', 'diya@example.com', '9000000002', 'Student', '2026-01-12'),
(3, 'Rohan Patel', 'rohan@example.com', '9000000003', 'Student', '2026-01-15'),
(4, 'Meera Nair', 'meera@example.com', '9000000004', 'Faculty', '2025-08-01'),
(5, 'Kiran Rao', 'kiran@example.com', '9000000005', 'Faculty', '2025-08-05'),
(6, 'Ananya Das', 'ananya@example.com', '9000000006', 'Student', '2026-02-01');

INSERT IGNORE INTO books
(book_id, isbn, title, author, category_id, publisher, publication_year,
 price, total_copies, copies_available, shelf_location) VALUES
(1, '9780131103627', 'The C Programming Language', 'Kernighan and Ritchie', 1, 'Pearson', 1988, 550.00, 4, 4, 'A-01'),
(2, '9780134494166', 'Clean Code', 'Robert C. Martin', 1, 'Pearson', 2008, 650.00, 3, 3, 'A-02'),
(3, '9789355423483', 'Python Basics', 'John Smith', 1, 'Tech Press', 2023, 450.00, 5, 5, 'A-03'),
(4, '9780078022159', 'Database System Concepts', 'Silberschatz', 2, 'McGraw Hill', 2019, 720.00, 4, 4, 'B-01'),
(5, '9780133970777', 'Fundamentals of Database Systems', 'Elmasri', 2, 'Pearson', 2016, 680.00, 3, 3, 'B-02'),
(6, '9780132126953', 'Computer Networks', 'Andrew Tanenbaum', 3, 'Pearson', 2010, 590.00, 3, 3, 'C-01'),
(7, '9780262033848', 'Introduction to Algorithms', 'Cormen', 1, 'MIT Press', 2009, 900.00, 2, 2, 'A-04'),
(8, '9780141439518', 'Pride and Prejudice', 'Jane Austen', 4, 'Penguin', 1813, 300.00, 2, 2, 'D-01'),
```

```
(9, '9780140449136', 'The Odyssey', 'Homer', 4, 'Penguin', 1996, 350.00, 2, 2, 'D-02'),
(10, '9780000000010', 'Basic Mathematics', 'S. Rao', 6, 'Academic Press', 2022, 400.00, 3, 3, 'E-01'),
(11, '9780000000011', 'Applied Statistics', 'M. Gupta', 6, 'Academic Press', 2021, 480.00, 2, 2, 'E-02'),
(12, '9780000000012', 'Introduction to Science', 'R. Kumar', 5, 'Learning House', 2020, 390.00, 3, 3, 'F-01');

-- 4. TRIGGERS
DROP TRIGGER IF EXISTS trg_issue_before_insert;
DROP TRIGGER IF EXISTS trg_issue_after_insert;
DROP TRIGGER IF EXISTS trg_issue_after_return;

DELIMITER $$

CREATE TRIGGER trg_issue_before_insert
BEFORE INSERT ON issues
FOR EACH ROW
BEGIN
    DECLARE available_count INT;
    SELECT copies_available INTO available_count
    FROM books WHERE book_id = NEW.book_id AND book_status = 'Active';

    IF available_count IS NULL OR available_count < 1 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Book does not exist, is archived, or has no available copies';
    END IF;

    IF NEW.due_date < NEW.issue_date THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Due date cannot be earlier than issue date';
    END IF;
END$$

CREATE TRIGGER trg_issue_after_insert
AFTER INSERT ON issues
FOR EACH ROW
BEGIN
    IF NEW.return_date IS NULL THEN
        UPDATE books
        SET copies_available = copies_available - 1
        WHERE book_id = NEW.book_id;
    END IF;

    INSERT INTO activity_log(activity_type, details)
    VALUES ('ISSUE_RECORDED',
            CONCAT('Issue #', NEW.issue_id, ', book #', NEW.book_id,
                   ', member #', NEW.member_id));
END$$

CREATE TRIGGER trg_issue_after_return
AFTER UPDATE ON issues
FOR EACH ROW
BEGIN
    IF OLD.return_date IS NULL AND NEW.return_date IS NOT NULL THEN
        UPDATE books
        SET copies_available = copies_available + 1
        WHERE book_id = NEW.book_id;

        INSERT INTO activity_log(activity_type, details)
        VALUES ('BOOK_RETURNED',
                CONCAT('Issue #', NEW.issue_id, ', book #', NEW.book_id,
                       ', member #', NEW.member_id));
```

```
    END IF;
END$$

DELIMITER ;

-- 4A. SAMPLE BORROWING AND RESERVATION DATA
-- These examples demonstrate returned, currently issued and overdue records.
-- The issue trigger adjusts copies_available for records that are still on loan.
INSERT IGNORE INTO issues
(issue_id, member_id, book_id, issue_date, due_date, return_date, issue_status)
VALUES
(1, 1, 1, '2026-10-20', '2026-11-03', '2026-10-30', 'Returned'),
(2, 2, 2, '2026-11-01', '2026-11-15', NULL, 'Issued'),
(3, 3, 4, '2026-10-15', '2026-10-29', NULL, 'Issued'),
(4, 4, 6, '2026-10-25', '2026-11-24', NULL, 'Issued'),
(5, 5, 5, '2026-10-01', '2026-10-31', '2026-11-04', 'Returned'),
(6, 1, 3, '2026-10-28', '2026-11-11', NULL, 'Issued'),
(7, 2, 8, '2026-10-10', '2026-10-24', '2026-10-22', 'Returned');

INSERT IGNORE INTO reservations
(reservation_id, member_id, book_id, reservation_status)
VALUES
(1, 3, 1, 'Pending'),
(2, 6, 4, 'Pending'),
(3, 4, 2, 'Fulfilled');

-- 5. REPORTING VIEWS
CREATE OR REPLACE VIEW vw_current_issues AS
SELECT i.issue_id, m.member_id, m.member_name, b.book_id, b.title,
       i.issue_date, i.due_date,
       DATEDIFF(CURRENT_DATE, i.due_date) AS days_overdue
FROM issues i
JOIN members m ON m.member_id = i.member_id
JOIN books b ON b.book_id = i.book_id
WHERE i.return_date IS NULL;

CREATE OR REPLACE VIEW vw_fine_report AS
SELECT i.issue_id, m.member_id, m.member_name, b.title,
       i.due_date, i.return_date,
       GREATEST(DATEDIFF(COALESCE(i.return_date, CURRENT_DATE), i.due_date), 0)
           AS late_days,
       GREATEST(DATEDIFF(COALESCE(i.return_date, CURRENT_DATE), i.due_date), 0)
           * 2.00 AS fine_amount
FROM issues i
JOIN members m ON m.member_id = i.member_id
JOIN books b ON b.book_id = i.book_id;

CREATE OR REPLACE VIEW vw_book_availability AS
SELECT b.book_id, b.title, c.category_name, b.total_copies,
       b.copies_available,
       (b.total_copies - b.copies_available) AS copies_on_loan
FROM books b
JOIN categories c ON c.category_id = b.category_id
WHERE b.book_status = 'Active';

-- 6. STORED PROCEDURES
DROP PROCEDURE IF EXISTS sp_issue_book;
DROP PROCEDURE IF EXISTS sp_return_book;
DROP PROCEDURE IF EXISTS sp_member_history;
```

```
DELIMITER $$

CREATE PROCEDURE sp_issue_book(
    IN p_member_id INT,
    IN p_book_id INT,
    IN p_due_date DATE
)
BEGIN
    DECLARE v_member_status VARCHAR(20);
    DECLARE v_member_type VARCHAR(20);
    DECLARE v_issue_date DATE;
    DECLARE v_due_date DATE;

    SELECT member_status, member_type
      INTO v_member_status, v_member_type
    FROM members
    WHERE member_id = p_member_id;

    IF v_member_status IS NULL OR v_member_status <> 'Active' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Member not found or inactive';
    END IF;

    SET v_issue_date = CURRENT_DATE;
    SET v_due_date = p_due_date;

    IF v_due_date IS NULL THEN
        IF v_member_type = 'Faculty' THEN
            SET v_due_date = DATE_ADD(v_issue_date, INTERVAL 30 DAY);
        ELSE
            SET v_due_date = DATE_ADD(v_issue_date, INTERVAL 14 DAY);
        END IF;
    END IF;

    INSERT INTO issues(member_id, book_id, issue_date, due_date)
    VALUES (p_member_id, p_book_id, v_issue_date, v_due_date);
END$$

CREATE PROCEDURE sp_return_book(IN p_issue_id INT)
BEGIN
    UPDATE issues
    SET return_date = CURRENT_DATE,
        issue_status = 'Returned'
    WHERE issue_id = p_issue_id
      AND return_date IS NULL;

    IF ROW_COUNT() = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Issue record not found or book already returned';
    END IF;
END$$

CREATE PROCEDURE sp_member_history(IN p_member_id INT)
BEGIN
    SELECT i.issue_id, b.title, i.issue_date, i.due_date,
           i.return_date, i.issue_status
    FROM issues i
    JOIN books b ON b.book_id = i.book_id
    WHERE i.member_id = p_member_id
    ORDER BY i.issue_date DESC;
```

```
END$$

DELIMITER ;

-- 7. USEFUL SQL REPORT QUERIES

-- Q1. Count members, books and issue records.
SELECT
    (SELECT COUNT(*) FROM members) AS total_members,
    (SELECT COUNT(*) FROM books) AS total_titles,
    (SELECT COUNT(*) FROM issues) AS total_issue_records;

-- Q2. Number of titles and available copies by category.
SELECT c.category_name, COUNT(b.book_id) AS title_count,
       COALESCE(SUM(b.copies_available), 0) AS available_copies
FROM categories c
LEFT JOIN books b ON b.category_id = c.category_id
GROUP BY c.category_id, c.category_name
ORDER BY title_count DESC;

-- Q3. Five most borrowed books.
SELECT b.title, COUNT(i.issue_id) AS times_borrowed
FROM books b
LEFT JOIN issues i ON i.book_id = b.book_id
GROUP BY b.book_id, b.title
ORDER BY times_borrowed DESC, b.title
LIMIT 5;

-- Q4. Members who have never borrowed a book.
SELECT m.member_id, m.member_name
FROM members m
LEFT JOIN issues i ON i.member_id = m.member_id
WHERE i.issue_id IS NULL;

-- Q5. Currently issued books.
SELECT * FROM vw_current_issues ORDER BY due_date;

-- Q6. Late returns and current overdue fines.
SELECT * FROM vw_fine_report
WHERE late_days > 0
ORDER BY fine_amount DESC;

-- Q7. Total fine by member.
SELECT member_id, member_name, SUM(fine_amount) AS total_fine
FROM vw_fine_report
GROUP BY member_id, member_name
HAVING SUM(fine_amount) > 0
ORDER BY total_fine DESC;

-- Q8. Borrowing comparison: students vs faculty.
SELECT m.member_type, COUNT(i.issue_id) AS total_borrowings
FROM members m
LEFT JOIN issues i ON i.member_id = m.member_id
GROUP BY m.member_type;

-- Q9. Rank members by borrowing count (MySQL 8.0+).
SELECT member_id, member_name, total_borrowed,
       RANK() OVER (ORDER BY total_borrowed DESC) AS borrowing_rank
FROM (
    SELECT m.member_id, m.member_name, COUNT(i.issue_id) AS total_borrowed
```

```
    FROM members m
    LEFT JOIN issues i ON i.member_id = m.member_id
    GROUP BY m.member_id, m.member_name
) AS borrowing_summary;

-- Q10. Most popular category.
SELECT c.category_name, COUNT(i.issue_id) AS total_borrowings
FROM categories c
JOIN books b ON b.category_id = c.category_id
LEFT JOIN issues i ON i.book_id = b.book_id
GROUP BY c.category_id, c.category_name
ORDER BY total_borrowings DESC
LIMIT 1;

-- Q11. Books with no available copies.
SELECT book_id, title, total_copies, copies_available
FROM books
WHERE copies_available = 0;

-- Q12. Reservations waiting for a book.
SELECT r.reservation_id, m.member_name, b.title, r.reservation_date
FROM reservations r
JOIN members m ON m.member_id = r.member_id
JOIN books b ON b.book_id = r.book_id
WHERE r.reservation_status = 'Pending'
ORDER BY r.reservation_date;

-- Q13. Call the member borrowing history procedure.
-- CALL sp_member_history(1);

-- Q14. Issue a book (use a valid member_id and book_id).
-- CALL sp_issue_book(1, 1, NULL); -- NULL uses 14 days for students, 30 for faculty.

-- Q15. Return a book (replace 1 with a currently open issue_id).
-- CALL sp_return_book(1);

-- 8. QUICK VERIFICATION
SELECT * FROM vw_book_availability;
SELECT * FROM activity_log ORDER BY activity_time DESC;
```